

Lab 12 – Relatório

Introdução

O laboratório envolve execução de serviços programando sockets TCP com uma escolha individual do serviço. No laboratório nós fizemos um socket que inclui o módulo cliente e módulo servidor utilizando o protocolo TCP. O serviço que escolhemos é um servidor de status, ou seja, o módulo cliente faz a requisição de uma consulta como: GET TIME, GET DATE, GET OS, GET HOSTNAME e para fechar o serviço o comando CLOSE. O módulo servidor, por sua vez, realiza essas consultas e devolve o resultado, sendo eles (exemplo): A data de hoje, o tempo atual e o sistema operacional da máquina.

Contextualização

O código do módulo servidor:

```
import socket
import platform
import datetime
import threading

HOST = '127.0.0.1' # Localhost
PORT = 65432      # Porta de comunicação

def get_system_status(command):
```

```

"""Processa o comando do cliente e retorna a resposta de status."""
command = command.strip().upper()

if command == 'GET TIME':
    return f"HORA ATUAL: {datetime.datetime.now().strftime('%H:%M:%S')}"

elif command == 'GET DATE':
    return f"DATA ATUAL: {datetime.datetime.now().strftime('%Y-%m-%d')}"

elif command == 'GET OS':
    return f"SISTEMA OPERACIONAL: {platform.system()} {platform.release()}"

elif command == 'GET HOSTNAME':
    return f"NOME DO HOST: {platform.node()}"

elif command == 'CLOSE':
    return "FECHANDO CONEXÃO"

else:
    return f"ERRO: Comando '{command}' desconhecido. Comandos válidos: GET TIME, GET DATE, GET OS, GET HOSTNAME, CLOSE."

def handle_client(conn, addr):
    """Função para lidar com a comunicação com um cliente específico."""
    print(f"[NOVA CONEXÃO] Conexão aceita de {addr}")

    try:
        while True:

```

```

# Recebe dados (máximo de 1024 bytes)

data = conn.recv(1024)

if not data:

    break # Conexão encerrada pelo cliente


comando_recebido = data.decode('utf-8').strip()

print(f"[{addr}] Comando recebido: '{comando_recebido}'")


# Processa o comando

resposta = get_system_status(comando_recebido)


# Envia a resposta de volta

conn.sendall(resposta.encode('utf-8'))


# Se o comando for 'CLOSE', encerra o loop de comunicação

if comando_recebido.upper() == 'CLOSE':

    break


except ConnectionResetError:

    print(f"[{addr}] Cliente desconectou inesperadamente.")

except Exception as e:

    print(f"[{addr}] Ocorreu um erro: {e}")

finally:

    print(f"[CONEXÃO FECHADA] Conexão com {addr} encerrada.")

    conn.close() # Fecha o socket de comunicação com o cliente


# Configuração do Servidor Principal

def start_server():

```

```

# Cria o socket TCP

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Opção para reutilizar a porta rapidamente após fechar

server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

try:

    # Liga o socket ao endereço e porta

    server_socket.bind((HOST, PORT))

    # Escuta por conexões (Permite até 5 conexões em espera)

    server_socket.listen(5)

    print(f"Servidor de Status escutando em {HOST}:{PORT}...")

    while True:

        # Aceita uma nova conexão (bloqueante)

        conn, addr = server_socket.accept()

        # Cria uma nova thread para lidar com o cliente, permitindo múltiplos
clientes

        client_thread = threading.Thread(target=handle_client, args=(conn, addr))

        client_thread.start()

        # Mostra o número de threads (clientes) ativos

        print(f"[ATIVO] Threads de clientes ativas: {threading.active_count() - 1}")

        # Subtrai 1 para não contar a thread principal

except KeyboardInterrupt:

    print("\nServidor sendo encerrado...")

```

```
except Exception as e:

    print(f"Erro no servidor: {e}")

finally:

    server_socket.close()


if __name__ == "__main__":

    start_server()
```

Código do módulo cliente:

```
import socket


HOST = '127.0.0.1' # O endereço IP do servidor
PORT = 65432      # A mesma porta usada pelo servidor


def start_client():

    # Cria o socket TCP

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

        try:

            # Conecta ao servidor

            s.connect((HOST, PORT))

            print(f"Conectado ao Servidor de Status em {HOST}:{PORT}")

            print("Comandos válidos: GET TIME, GET DATE, GET OS, GET HOSTNAME, CLOSE")


        while True:

            # Pede o comando ao usuário

            command = input("Digite o comando: ").strip()
```

```
if not command:
    continue

# 1. Envia o comando
s.sendall(command.encode('utf-8'))

# 2. Recebe a resposta do servidor
data = s.recv(1024)
if not data:
    print("Servidor fechou a conexão.")
    break

response = data.decode('utf-8')
print(f"RESPOSTA DO SERVIDOR: {response}")

# Se o servidor indicar que está fechando, o cliente também encerra
if command.upper() == 'CLOSE' or "FECHANDO CONEXÃO" in response:
    break

except ConnectionRefusedError:
    print("ERRO: Servidor não encontrado ou não está escutando.")
except Exception as e:
    print(f"Ocorreu um erro: {e}")

print("Cliente encerrado.")

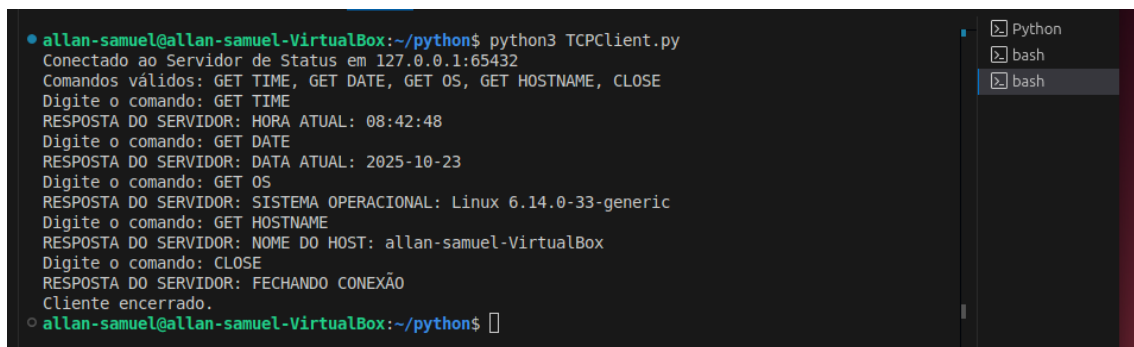
if __name__ == "__main__":
```

start_client()

Atividades

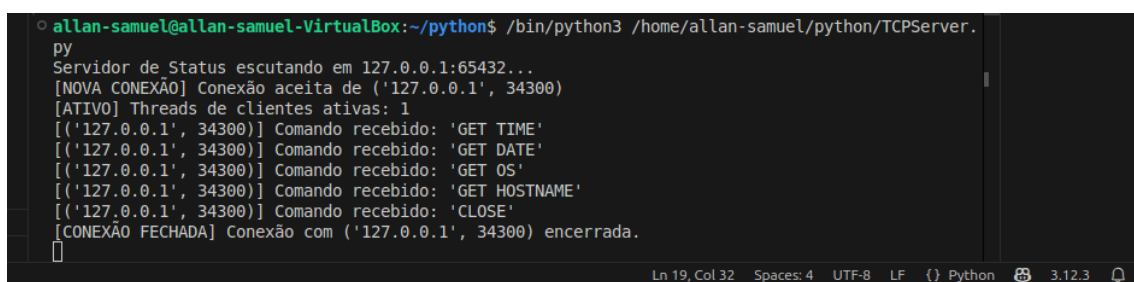
Nas atividades, foi feito, primeiramente, a ativação do módulo servidor, logo após abriu-se um novo terminal na mesma máquina e ativou-se o módulo cliente. Logo depois, os comandos do código foram utilizados para realizar essa comunicação via SOCKET, foram utilizados todos os comandos possíveis: GET TIME, GET DATE, GET OS, GET HOSTNAME e, para finalizar o serviço, o comando CLOSE. Os prints abaixo mostram isso com veracidade.

PRINT 1:



```
allan-samuel@allan-samuel-VirtualBox:~/python$ python3 TCPClient.py
Conectado ao Servidor de Status em 127.0.0.1:65432
Comandos válidos: GET TIME, GET DATE, GET OS, GET HOSTNAME, CLOSE
Digite o comando: GET TIME
RESPOSTA DO SERVIDOR: HORA ATUAL: 08:42:48
Digite o comando: GET DATE
RESPOSTA DO SERVIDOR: DATA ATUAL: 2025-10-23
Digite o comando: GET OS
RESPOSTA DO SERVIDOR: SISTEMA OPERACIONAL: Linux 6.14.0-33-generic
Digite o comando: GET HOSTNAME
RESPOSTA DO SERVIDOR: NOME DO HOST: allan-samuel-VirtualBox
Digite o comando: CLOSE
RESPOSTA DO SERVIDOR: FECHANDO CONEXÃO
Cliente encerrado.
allan-samuel@allan-samuel-VirtualBox:~/python$
```

PRINT 2:



```
allan-samuel@allan-samuel-VirtualBox:~/python$ /bin/python3 /home/allan-samuel/python/TCPServer.py
Servidor de Status escutando em 127.0.0.1:65432...
[NOVA CONEXÃO] Conexão aceita de ('127.0.0.1', 34300)
[ATIVO] Threads de clientes ativas: 1
[('127.0.0.1', 34300)] Comando recebido: 'GET TIME'
[('127.0.0.1', 34300)] Comando recebido: 'GET DATE'
[('127.0.0.1', 34300)] Comando recebido: 'GET OS'
[('127.0.0.1', 34300)] Comando recebido: 'GET HOSTNAME'
[('127.0.0.1', 34300)] Comando recebido: 'CLOSE'
[CONEXÃO FECHADA] Conexão com ('127.0.0.1', 34300) encerrada.
```

Prints do wireshark para estabelecimento de conexões:

PRINT 3:

/tmp/wireshark_loC9PTE3.pcapng 36 total packets, 22 shown

No.	Time	Source	Destination	Protocol	Length	Info
11	13.002207785	127.0.0.1	127.0.0.1	TCP	74	54476 → 65432 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1540878925 TSecr=0 WS=128 Frame 11: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface lo, id 0 Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00) Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 0, Len: 0
12	13.002226320	127.0.0.1	127.0.0.1	TCP	74	65432 → 54476 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1540878925 TSecr=1540878925 WS=128 Frame 12: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface lo, id 0 Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00) Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 Transmission Control Protocol, Src Port: 65432, Dst Port: 54476, Seq: 0, Ack: 1, Len: 0
13	13.002241887	127.0.0.1	127.0.0.1	TCP	66	54476 → 65432 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1540878925 TSecr=1540878925 Frame 13: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0 Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00) Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 1, Ack: 1, Len: 0

Conseguimos verificar com o print acima que o cliente envia um SYN, o servidor responde com SYN-ACK e o cliente confirma com ACK.

Print do wireshark para troca de mensagens:

PRINT 4:

/tmp/wireshark_loC9PTE3.pcapng 36 total packets, 22 shown

```
No.      Time      Source      Destination      Protocol Length Info
18 15.519045403 127.0.0.1 127.0.0.1 TCP 74 54476 → 65432 [PSH,
ACK] Seq=1 Ack=1 Win=65536 Len=8 TSval=1540881442 TSecr=1540878925
Frame 18: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 1, Ack: 1, Len: 8
Data (8 bytes)
0000 47 45 54 20 54 49 4d 45 GET TIME
No.      Time      Source      Destination      Protocol Length Info
19 15.519061100 127.0.0.1 127.0.0.1 TCP 66 65432 → 54476 [ACK]
Seq=1 Ack=9 Win=65536 Len=0 TSval=1540881442 TSecr=1540881442
Frame 19: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 65432, Dst Port: 54476, Seq: 1, Ack: 9, Len: 0
No.      Time      Source      Destination      Protocol Length Info
20 15.519251643 127.0.0.1 127.0.0.1 TCP 86 65432 → 54476 [PSH,
ACK] Seq=1 Ack=9 Win=65536 Len=20 TSval=1540881442 TSecr=1540881442
Frame 20: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 65432, Dst Port: 54476, Seq: 1, Ack: 9, Len: 20
Data (20 bytes)
0000 48 4f 52 41 20 41 54 55 41 4c 3a 20 30 38 3a 34 HORA ATUAL: 08:4
0010 39 3a 32 38 9:28
No.      Time      Source      Destination      Protocol Length Info
21 15.519264053 127.0.0.1 127.0.0.1 TCP 66 54476 → 65432 [ACK]
Seq=9 Ack=21 Win=65536 Len=0 TSval=1540881442 TSecr=1540881442
Frame 21: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 9, Ack: 21, Len: 0
No.      Time      Source      Destination      Protocol Length Info
22 17.262891881 127.0.0.1 127.0.0.1 TCP 74 54476 → 65432 [PSH,
ACK] Seq=9 Ack=21 Win=65536 Len=8 TSval=1540883186 TSecr=1540881442
Frame 22: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 9, Ack: 21, Len: 8
Data (8 bytes)
0000 47 45 54 20 44 41 54 45 GET DATE
No.      Time      Source      Destination      Protocol Length Info
23 17.263036688 127.0.0.1 127.0.0.1 TCP 88 65432 → 54476 [PSH,
ACK] Seq=21 Ack=17 Win=65536 Len=22 TSval=1540883186 TSecr=1540883186
Frame 23: 88 bytes on wire (704 bits), 88 bytes captured (704 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 65432, Dst Port: 54476, Seq: 21, Ack: 17, Len: 22
Data (22 bytes)
0000 44 41 54 41 20 41 54 55 41 4c 3a 20 32 30 32 35 DATA ATUAL: 2025
0010 2d 31 30 2d 32 33 -10-23
No.      Time      Source      Destination      Protocol Length Info
24 17.263086879 127.0.0.1 127.0.0.1 TCP 66 54476 → 65432 [ACK]
Seq=17 Ack=43 Win=65536 Len=0 TSval=1540883186 TSecr=1540883186
Frame 24: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 17, Ack: 43, Len: 0
No.      Time      Source      Destination      Protocol Length Info
25 18.763225113 127.0.0.1 127.0.0.1 TCP 72 54476 → 65432 [PSH,
ACK] Seq=17 Ack=43 Win=65536 Len=6 TSval=1540884686 TSecr=1540883186
Frame 25: 72 bytes on wire (576 bits), 72 bytes captured (576 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 17, Ack: 43, Len: 6
Data (6 bytes)
0000 47 45 54 20 4f 53 GET OS
No.      Time      Source      Destination      Protocol Length Info
26 18.763473536 127.0.0.1 127.0.0.1 TCP 110 65432 → 54476 [PSH,
ACK] Seq=43 Ack=23 Win=65536 Len=44 TSval=1540884686 TSecr=1540884686
Frame 26: 110 bytes on wire (880 bits), 110 bytes captured (880 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 65432, Dst Port: 54476, Seq: 43, Ack: 23, Len: 44
Data (44 bytes)
```

PRINT 5:

/tmp/wireshark_loC9PTE3.pcapng 36 total packets, 22 shown

```
0000 53 49 53 54 45 4d 41 20 4f 50 45 52 41 43 49 4f  SISTEMA OPERACIO
0010 4e 41 4c 3a 20 4c 69 6e 75 78 20 36 2e 31 34 2e  NAL: Linux 6.14.
0020 30 2d 33 33 2d 67 65 6e 65 72 69 63  0-33-generic
No.    Time                Source                Destination                Protocol Length Info
    27 18.763515948      127.0.0.1          127.0.0.1                  TCP                66      54476 → 65432 [ACK]
Seq=23 Ack=87 Win=65536 Len=0 TSval=1540884686 TSecr=1540884686
Frame 27: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 23, Ack: 87, Len: 0
No.    Time                Source                Destination                Protocol Length Info
    28 22.597126575      127.0.0.1          127.0.0.1                  TCP                78      54476 → 65432 [PSH,
ACK] Seq=23 Ack=87 Win=65536 Len=12 TSval=1540888520 TSecr=1540884686
Frame 28: 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 23, Ack: 87, Len: 12
Data (12 bytes)
0000 47 45 54 20 48 4f 53 54 4e 41 4d 45  GET HOSTNAME
No.    Time                Source                Destination                Protocol Length Info
    29 22.597366576      127.0.0.1          127.0.0.1                  TCP                103     65432 → 54476 [PSH,
ACK] Seq=87 Ack=35 Win=65536 Len=37 TSval=1540888520 TSecr=154088520
Frame 29: 103 bytes on wire (824 bits), 103 bytes captured (824 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 65432, Dst Port: 54476, Seq: 87, Ack: 35, Len: 37
Data (37 bytes)
0000 4e 4f 4d 45 20 44 4f 20 48 4f 53 54 3a 20 61 6c  NOME DO HOST: al
0010 6c 61 6e 2d 73 61 6d 75 65 6c 2d 56 69 72 74 75  lan-samuel-Virtu
0020 61 6c 42 6f 78  alBox
No.    Time                Source                Destination                Protocol Length Info
    30 22.597377122      127.0.0.1          127.0.0.1                  TCP                66      54476 → 65432 [ACK]
Seq=35 Ack=124 Win=65536 Len=0 TSval=1540888520 TSecr=1540888520
Frame 30: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 35, Ack: 124, Len: 0
No.    Time                Source                Destination                Protocol Length Info
    31 25.616967259      127.0.0.1          127.0.0.1                  TCP                71      54476 → 65432 [PSH,
ACK] Seq=35 Ack=124 Win=65536 Len=5 TSval=1540891540 TSecr=1540888520
Frame 31: 71 bytes on wire (568 bits), 71 bytes captured (568 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 35, Ack: 124, Len: 5
Data (5 bytes)
0000 43 4c 4f 53 45  CLOSE
No.    Time                Source                Destination                Protocol Length Info
    32 25.617132094      127.0.0.1          127.0.0.1                  TCP                83      65432 → 54476 [PSH,
ACK] Seq=124 Ack=40 Win=65536 Len=17 TSval=1540891540 TSecr=1540891540
Frame 32: 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 65432, Dst Port: 54476, Seq: 124, Ack: 40, Len: 17
Data (17 bytes)
0000 46 45 43 48 41 4e 44 4f 20 43 4f 4e 45 58 c3 83  FECHANDO CONEX..
0010 4f  O
No.    Time                Source                Destination                Protocol Length Info
    33 25.617142990      127.0.0.1          127.0.0.1                  TCP                66      54476 → 65432 [ACK]
Seq=40 Ack=141 Win=65536 Len=0 TSval=1540891540 TSecr=1540891540
Frame 33: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 40, Ack: 141, Len: 0
```

É possível ver, com os prints acima, que a cada comando dado, ou seja, o GET TIME, GET DATE, GET OS GET HOSTNAME e CLOSE, há um PSH+ACK e a cada resposta tem um ACK de confirmação.

Prints do wireshark para o encerramento do SOCKET.

PRINT 6:

/tmp/wireshark_loC9PTE3.pcapng 36 total packets, 22 shown

No.	Time	Source	Destination	Protocol	Length	Info
34	25.617225548	127.0.0.1	127.0.0.1	TCP	66	54476 → 65432 [FIN,
ACK] Seq=40 Ack=141 Win=65536 Len=0 TSval=1540891540 TSecr=1540891540						
Frame 34: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 40, Ack: 141, Len: 0						
35	25.617909578	127.0.0.1	127.0.0.1	TCP	66	65432 → 54476 [FIN,
ACK] Seq=141 Ack=41 Win=65536 Len=0 TSval=1540891541 TSecr=1540891540						
Frame 35: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 65432, Dst Port: 54476, Seq: 141, Ack: 41, Len: 0						
36	25.617929155	127.0.0.1	127.0.0.1	TCP	66	54476 → 65432 [ACK]
Seq=41 Ack=142 Win=65536 Len=0 TSval=1540891541 TSecr=1540891541						
Frame 36: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 54476, Dst Port: 65432, Seq: 41, Ack: 142, Len: 0						

É possível verificar, no print acima, que quando há o comando CLOSE, ou seja, o encerramento do programa, há os pacotes FIN e ACK, sinalizando, assim, o fim da conexão.

Conclusão

Um trabalho extremamente prazeroso de se fazer, uma vez que colocar em prática o que foi aprendido na teoria é a melhor maneira de se aprender. É possível verificar que todo o processo foi um sucesso, desde a criação dos módulos, ou seja, do SOCKET até o fim do processo, ou seja, o encerramento do SOCKET. Dessa maneira, apesar da complexidade, o aprendizado valeu a pena, verificar no wireshark, mais especificamente no loopback, todo o processo sendo realizado em tempo real, torna tudo mais interessante.